



PERTEMUAN XI

MODEL DEPLOYMENT (STREAMLIT)

11.1. TUJUAN PEMBELAJARAN

- A. Memahami perbedaan paradigma deployment antara Web Service (Flask) dan Data Application (Streamlit).
- B. Menggunakan pustaka Streamlit untuk membuat antarmuka web interaktif berbasis Python.
- C. Membuat input widget (form, slider, dropdown) untuk menerima data dari pengguna.
- D. Mengintegrasikan model machine learning ke dalam dashboard untuk melakukan prediksi secara visual.

11.2. DASAR TEORI

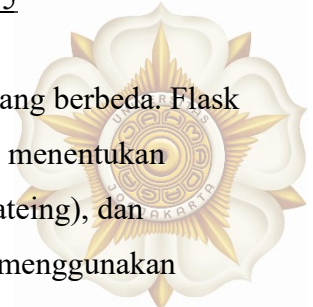
- **Mengenal Streamlit**

Streamlit adalah framework Python open-source yang dirancang khusus untuk Data Scientist dan Machine Learning Engineer. Streamlit memungkinkan pembuatan aplikasi web yang interaktif dan data-centric dalam waktu yang sangat singkat.

Berbeda dengan Flask yang mengharuskan kita menulis logika routing dan template HTML secara terpisah, Streamlit mengubah skrip data menjadi aplikasi web. Setiap kali pengguna berinteraksi dengan widget (misal: menggeser slider), Streamlit akan menjalankan ulang skrip dari atas ke bawah (rerun) untuk memperbarui tampilan.

- **Perbedaan Flask dan Streamlit**

Meskipun keduanya digunakan untuk membuat aplikasi berbasis web dengan Python, filosofi dan cara kerjanya sangat berbeda. Secara konsep kedua hal tersebut memiliki perbedaan di filosofi pengembangan dan juga arsitektur yang digunakan.



Secara filosofi pengembangan, flask dan streamlit memiliki tujuan yang berbeda. Flask memberikan kendali penuh pada pengembangan. Pengembang harus menentukan struktur URL (routing), bagaimana halaman HTML dirender (templateing), dan bagaimana data dikirim (HTTP Request/Response). Pengembangan menggunakan filosofi Flask sangat cocok untuk membangun sistem produksi yang kompleks atau API yang akan dikonsumsi oleh aplikasi lain. Berbeda dengan Streamlit, kecepatan pengembangan menjadi fokus utama. Streamlit “menyembunyikan” kompleksitas web (HTML/CSS/JS) di balik sintaks Python sederhana. Cocok untuk Data Scientist yang ingin mempresentasikan hasil model, membuat prototype cepat, atau dashboard internal tanpa perlu belajar front-end engineering.

Secara arsitektur, flask menggunakan model request-response dan streamlit menggunakan model script rerun. Arsitektur request-response akan berjalan ketika ada permintaan (request) ke URL tertentu. Variabel yang terbuat ketika menjalankan eksekusi request akan hilang setelah request selesai, kecuali disimpan di dalam database atau session. Sementara itu, Streamlit akan menjalankan seluruh skrip Python dari baris paling atas hingga paling bawah setiap kali pengguna berinteraksi dengan widget (misal menggeser slider atau mengetik di text box). Oleh karena itu, variabel biasa akan ter-reset, sehingga streamlit menyediakan fitur *Session State* dan *Caching* untuk mengatasi hal tersebut.

Fitur	Flask	Streamlit
Fokus Utama	Backend API & Web Development Umum	Data Dashboard & Prototyping Cepat
Front End	Perlu HTML/CSS/JS manual	Otomatis
Fleksibilitas	Sangat Tinggi	Terbatas pada pola Data App
Pengguna	Developer & Sistem Lain	End-User



- **Komponen Utama Streamlit**

Streamlit menyediakan berbagai fungsi ("komponen") yang dikategorikan berdasarkan kegunaannya. Berikut adalah komponen vital yang akan sering digunakan dalam Data Mining:

1. **Output dan Text Display (Menampilkan Informasi)**

Komponen ini menggantikan fungsi `print()` di Python biasa atau tag HTML seperti:

`<p>`, `<h1>`, dll.

- `st.write()`

Fungsi ini digunakan untuk menampilkan teks, dataframe, atau grafik secara otomatis tergantung dengan inputnya.

- `st.title()`, `st.header()`, `st.subheader()`

Fungsi ini digunakan untuk membuat hierarki judul untuk mengganti tag HTML `<h1>`, `<h2>`, dst.

- `st.markdown()`

Fungsi ini digunakan untuk menampilkan teks dengan format Markdown (bold, italic, link, latex).

- `st.metrics()`

Fungsi ini sangat berguna untuk dashboard, menampilkan angka besar dengan indikator delta (naik/turun). Contoh: menampilkan akurasi model.

2. **Input Widgets (Menerima Data Pengguna)**

Input widget adalah alat interaksi utama. Nilai kembali (return value) dari widget adalah data yang diinputkan pengguna.

- `st.text_input()`, `st.number_input()`

Fungsi ini digunakan untuk input data manual.

- `st.slider()`, `st.select_slider()`

Fungsi ini digunakan untuk input angka dalam rentang tertentu (cocok digunakan untuk hyperparameter tuning).

- `st.selectbox()`, `st.multiselect()`

Fungsi ini digunakan untuk membuat menu dropdown.

- `st.file_uploader()`



Fungsi ini digunakan untuk mengunggah file (CSV, Gambar, text) yang akan diproses model.

3. Layouts & Container (Mengatur Tata Letak)

Secara default, Streamlit akan menumpuk elemen dari atas ke bawah. Komponen ini membantu merapikan tampilan:

- `st.sidebar`

Fungsi ini digunakan untuk menampilkan sidebar dan dapat digunakan untuk menaruh kontrol input agar tidak mengganggu tampilan utama.

- `st.column()`

Fungsi ini dapat membagi layar menjadi beberapa kolom vertikal (seperti grid).

- `st.tabs()`

Fungsi ini dapat digunakan untuk membuat navigasi tab dalam satu halaman.

- `st.expander()`

Fungsi ini dapat digunakan untuk membuat area yang bisa diklik untuk dibuka-tutup.

4. Caching (Optimasi Performa)

Oleh karena Streamlit menjalankan ulang (rerun) skrip setiap kali ada interaksi, proses berat seperti membuat model Machine Learning atau membaca dataset besar akan membuat aplikasi menjadi lambat jika dilakukan berulang-ulang.

Untuk mengatasi hal tersebut, dekorator `@st.cache_resource` dapat digunakan secara khusus untuk menyimpan objek secara global agar hanya membuat objek tersebut sekali saja. Contoh penggunaan dekorator `@st.cache_resource` seperti berikut.

```
1 # Contoh implementasi Caching nanti di Langkah Percobaan
2 @st.cache_resource
3 def load_model():
4     return joblib.load('iris_model.pkl')
5
6 model = load_model() # Model hanya di-load sekali, tidak berulang-ulang
```

Gambar 0.1 Contoh penggunaan dekorator `st.cache_resource`



Dokumentasi komponen dan cara-cara penggunaan secara lengkap dapat diakses di website Streamlit.

<https://docs.streamlit.io/>

11.3. ALAT DAN BAHAN

- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**
 - Python3
 - Text Editor / IDE. Catatan: Tidak disarankan menggunakan Jupyter Notebook untuk menjalankan server Flask.
 - Browser
- **Pustaka Python**
 - streamlit
 - pandas
 - joblib atau pickle

11.4. LANGKAH PERCOBAAN

Pada pembuatan modul praktikum, environment yang digunakan adalah sistem operasi Linux Ubuntu di dalam WSL, Python 3.12, dan memiliki akses GPU. Oleh karena ini beberapa perintah dapat disesuaikan dengan target environment deployment masing-masing.

Dalam membuat proyek python alangkah baiknya menggunakan virtual environment yang sesuai dengan environment produksi yang ada di server. Pembuatan environment ini juga dapat membantu dalam manajemen pustaka agar tidak terjadi konflik dengan pustaka global lainnya. Untuk mempelajari lebih lanjut mengenai environment di python Anda dapat membaca referensi berikut:

- (vanilla) <https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/>



- (UV Astral) <https://docs.astral.sh/uv/>
- (Miniconda) <https://www.anaconda.com/docs/getting-started/miniconda/main>

Dalam percobaan ini, kita akan mencoba membuat sebuah aplikasi. Lakukan langkah-langkah berikut:

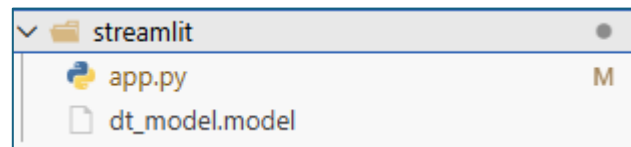
1. Unduh pustaka

Unduh streamlit, pandas, dan joblib. Berikut adalah kode instalasi dengan menggunakan PIP.

```
pip install streamlit pandas joblib
```

2. Struktur Proyek

Buatlah file app.py yang akan digunakan untuk menjalankan aplikasi Streamlit. Masukkan model yang telah dibuat pada pertemuan sebelumnya (dt_model.model) pada direktori yang sama seperti Gambar 0.1.



Gambar 0.1 Struktur Proyek Praktikum Streamlit

3. Jalankan Aplikasi

Jalankan aplikasi streamlit dengan menjalankan perintah berikut pada direktori dimana app.py streamlit berada. Jika berhasil maka tampilan terminal akan terlihat seperti Gambar 0.2. Oleh karena streamlit menggunakan arsitektur re-run, maka perubahan yang ada di dalam file app.py akan terimplementasikan secara langsung di browser setelah browser tersebut di refresh, sehingga terminal dapat dibiarkan saja ketika development aplikasi.

```
streamlit run app.py
```



```
[15:22:36] (.venv) ~/test/streamlit
$ streamlit run app.py
```

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8501>
 Network URL: <http://172.21.212.222:8501>

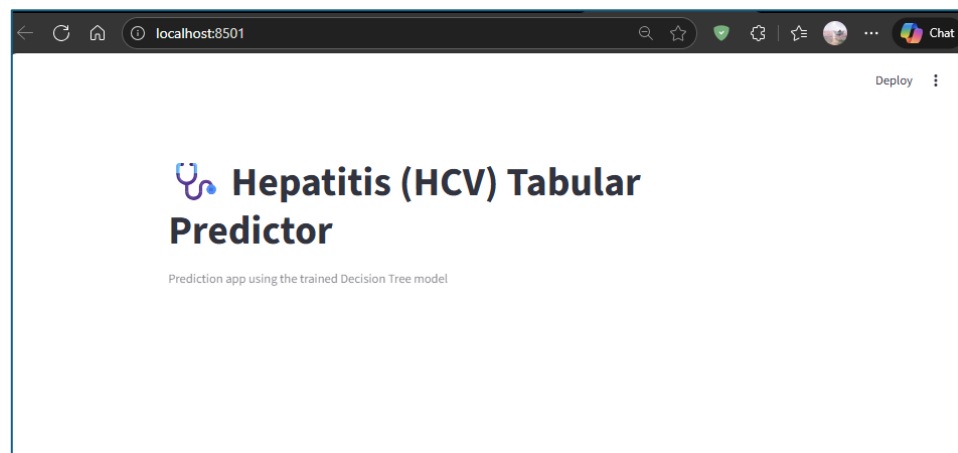
Gambar 0.2 Aplikasi Streamlit berhasil dijalankan

4. Menambahkan konfigurasi halaman web, title, dan caption

Menambahkan konfigurasi halaman web, title, dan caption dapat dilakukan dengan fungsi `set_page_config`, `title`, dan juga `caption`. Tambahkan kode Gambar 0.3 pada skrip `app.py`. Jika aplikasi streamlit dibuka, maka akan tertampil judul dan caption seperti Gambar 0.4.

```
1 import streamlit as st
2
3 st.set_page_config(
4     page_title="HCV Tabular Predictor",
5     page_icon="🩺",
6     layout="centered"
7 )
8
9 st.title("🩺 Hepatitis (HCV) Tabular Predictor")
10 st.caption("Prediction app using the trained Decision Tree model")
```

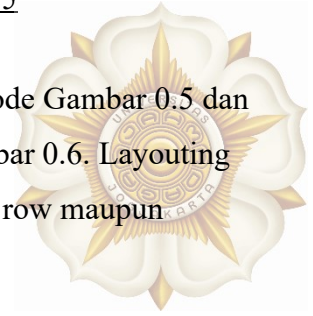
Gambar 0.3 Menambahkan Konfigurasi Web, Title, Caption



Gambar 0.4 Halaman yang tertampil di Streamlit

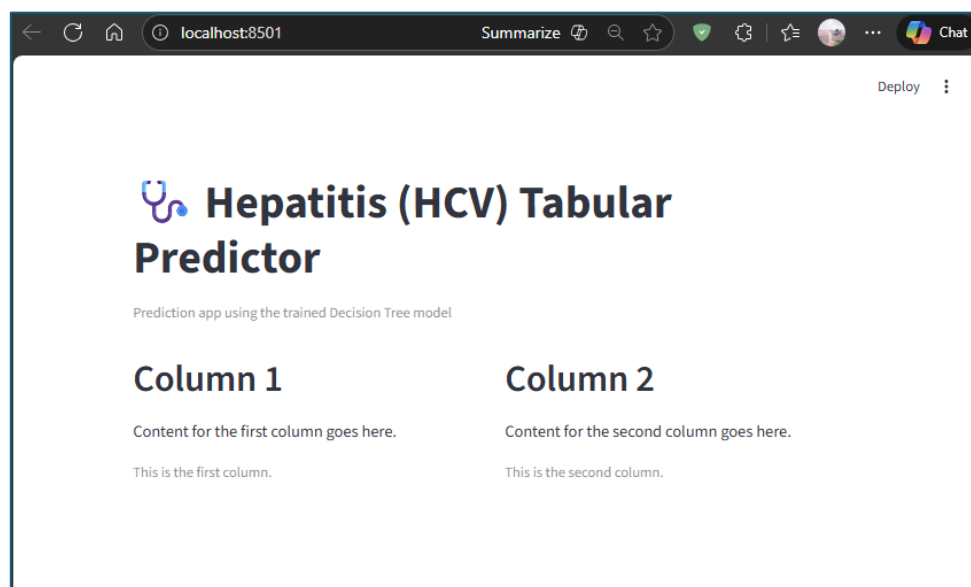
5. Membuat layout dasar

Pada tahap ini akan dibuat layout berupa dua kolom. Tambahkan kode Gambar 0.5 dan refresh halaman web sehingga halaman akan tertampil seperti Gambar 0.6. Layouting tidak terbatas pada column saja, tetapi bisa dikombinasikan dengan row maupun dilakukan nested row ataupun column.



```
1 top_col1, top_col2 = st.columns(2)
2
3 with top_col1:
4     st.header("Column 1")
5     st.write("Content for the first column goes here.")
6     st.caption("This is the first column.")
7 with top_col2:
8     st.header("Column 2")
9     st.write("Content for the second column goes here.")
10    st.caption("This is the second column.")
```

Gambar 0.5 Layouting 2 kolom



Gambar 0.6 Hasil layouting 2 kolom

6. Membuat Sidebar

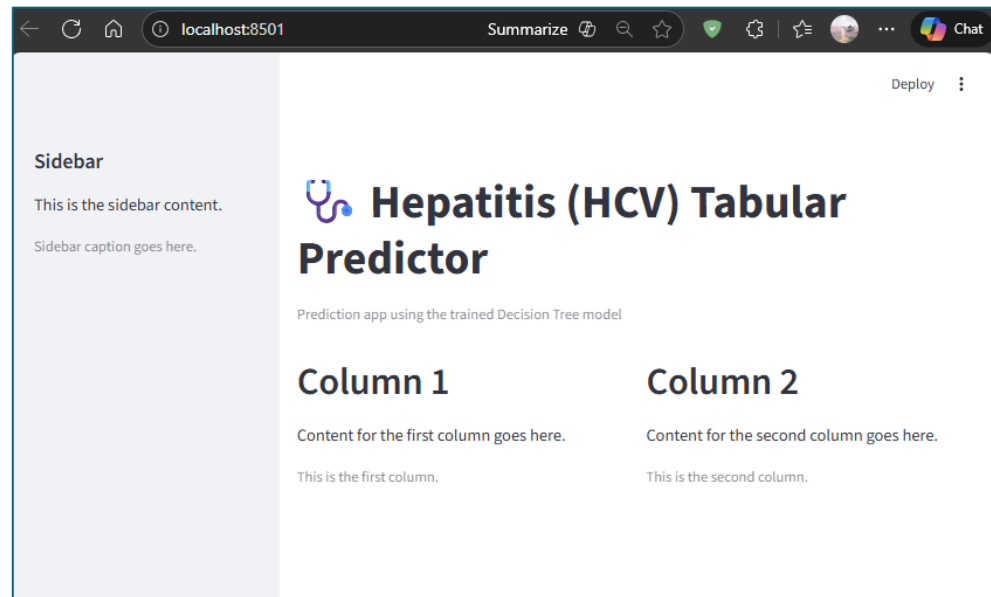
Untuk membuat sidebar sederhana, tambahkan kode Gambar 0.7. Lakukan refresh pada halaman web, maka sidebar akan terbentuk seperti Gambar 0.8.



```

1 st.sidebar.header("Sidebar")
2 st.sidebar.write("This is the sidebar content.")
3 st.sidebar.caption("Sidebar caption goes here.")
    
```

Gambar 0.7 Kode Streamlit Sidebar



Gambar 0.8 Halaman Streamlit diberikan sidebar

7. Melakukan integrasi dengan model AI

Buat variabel `MODEL_PATH` dan definisikan lokasi model berada. Letakkan kode berikut pada bagian atas. Sebagai contoh pembuatan variabel seperti Gambar 0.9.

```

1 MODEL_PATH = "dt_model.model"
    
```

Gambar 0.9 Model Path

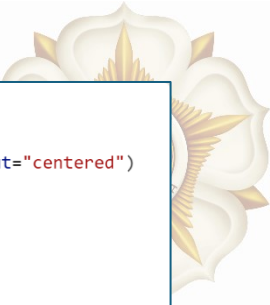
Tambahkan fungsi untuk memuat model seperti pada pertemuan sebelumnya. Berbeda sedikit dengan pertemuan sebelumnya, Pada streamlit, model dimuat dengan bantuan dekorator `cache` streamlit. Hal tersebut dilakukan agar model dapat dimuat dengan cepat di aplikasi streamlit. Kode memuat model dapat dilihat pada Gambar 0.10.



```
1 @st.cache_resource(show_spinner=True)
2 def load_model(path: str = MODEL_PATH):
3     return joblib.load(path)
```

Gambar 0.10 Load model dengan cache

Setelah itu buat logika aplikasi untuk manajemen state beserta fungsi bantuan lainnya yang telah dibuat pada pertemuan sebelumnya. Lengkapi kode yang dibuat menjadi seperti Gambar 0.11. Konstanta default yang didefinisikan pada baris ke 9 tersebut akan digunakan sebagai nilai untuk mereset ulang formulir dari aplikasi. Nilai default ini juga akan di muat ke dalam session pada baris ke 25.



```

1  st.set_page_config(page_title="HCV Tabular Predictor", page_icon="🦋", layout="centered")
2
3  st.title("🦋 Hepatitis (HCV) Tabular Predictor")
4  st.caption("Prediction app using the trained Decision Tree model")
5
6  MODEL_PATH = "dt_model.model"
7
8  # Default input values
9  DEFAULTS = {
10     "age": 30,
11     "sex": "Male",
12     "alb": 39.0,
13     "alp": 100.0,
14     "alt": 28.0,
15     "ast": 143.0,
16     "bil": 15.0,
17     "che": 5.0,
18     "chol": 4.0,
19     "crea": 72.0,
20     "ggt": 400.0,
21     "prot": 70.0,
22 }
23
24 # Initialize session state with defaults if missing
25 for k, v in DEFAULTS.items():
26     if k not in st.session_state:
27         st.session_state[k] = v
28
29 @st.cache_resource(show_spinner=True)
30 def load_model(path: str = MODEL_PATH):
31     return joblib.load(path)
32
33 def _to_float(x: Optional[float]) -> float:
34     try:
35         return float(x) if x is not None else float('nan')
36     except Exception:
37         return float('nan')
38
39 def _encode_sex(x: Optional[str]) -> int:
40     if x is None:
41         return 0
42     s = str(x).strip().lower()
43     if s in ("m", "male", "1", "true"):
44         return 1
45     if s in ("f", "female", "0", "false"):
46         return 0
47     try:
48         return int(float(s))
49     except Exception:

```

Gambar 0.11 Manajemen State dan Fungsi bantuan

Tambahkan tombol yang dapat digunakan untuk mereset nilai formulir ke nilai default. Kode implementasi hal tersebut akan terlihat seperti Gambar 0.12.



```

1 # Top-level Reset button must run before widgets are created
2 top_col1, top_col2 = st.columns([1, 1])
3 with top_col2:
4     if st.button("Reset to defaults", type="secondary"):
5         for k, v in DEFAULTS.items():
6             st.session_state[k] = v
7         st.rerun()

```

Gambar 0.12 Tombol Reset Formulir

Buatlah halaman formulir yang akan digunakan untuk menerima input. Dengan memanfaatkan ilmu layout yang telah dipelajari sebelumnya, praktikan dapat berekperimen untuk membuat formulir input. Sebagai contoh, kode formulir pada streamlit akan terlihat seperti Gambar 0.13. Pada baris ke 19, variabel submitted akan digunakan untuk melihat state tombol formulir apakah ditekan atau tidak. Variabel ini akan digunakan di tahap selanjutnya. Implementasi kode diatas hingga saat ini akan terlihat seperti Gambar 0.15.

```

1 with st.form("input-form"):
2     st.subheader("Enter Patient Lab Values")
3     col1, col2 = st.columns(2)
4     with col1:
5         age = st.number_input("Age", min_value=0, max_value=120, value=st.session_state["age"], step=1, key="age")
6         sex = st.selectbox("Sex", options=["Male", "Female"], index=0 if st.session_state["sex"] == "Male" else 1, key="sex")
7         alb = st.number_input("ALB (Albumin)", min_value=0.0, value=st.session_state["alb"], step=0.1, key="alb")
8         alp = st.number_input("ALP (Alkaline Phosphatase)", min_value=0.0, value=st.session_state["alp"], step=0.1, key="alp")
9         alt = st.number_input("ALT (Alanine Transaminase)", min_value=0.0, value=st.session_state["alt"], step=0.1, key="alt")
10        ast = st.number_input("AST (Aspartate Transaminase)", min_value=0.0, value=st.session_state["ast"], step=0.1, key="ast")
11    with col2:
12        bil = st.number_input("BIL (Bilirubin)", min_value=0.0, value=st.session_state["bil"], step=0.1, key="bil")
13        che = st.number_input("CHE (Cholinesterase)", min_value=0.0, value=st.session_state["che"], step=0.1, key="che")
14        chol = st.number_input("CHOL (Cholesterol)", min_value=0.0, value=st.session_state["chol"], step=0.1, key="chol")
15        crea = st.number_input("CREA (Creatinine)", min_value=0.0, value=st.session_state["crea"], step=0.1, key="crea")
16        ggt = st.number_input("GGT (Gamma-Glutamyl Transferase)", min_value=0.0, value=st.session_state["ggt"], step=0.1, key="ggt")
17        prot = st.number_input("PROT (Total Protein)", min_value=0.0, value=st.session_state["prot"], step=0.1, key="prot")
18
19    submitted = st.form_submit_button("Predict")

```

Gambar 0.13 Kode Formulir Streamlit

Buatlah output untuk menampilkan hasil prediksi beserta logika prediksinya seperti Gambar 0.14. Variabel submitted yang dibuat di tahap sebelumnya dipakai sebagai logika menampilkan output sehingga ketika tidak ada input, laman output tidak tertampil. Fungsi with spinner seperti pada baris 2 Gambar 0.14 digunakan untuk menampilkan UI loading. Di dalam fungsi tersebut terdapat aksi pembuatan model, prediksi (inference), dan pembaca hasil output. Logika ini tidak jauh berbeda dari

pertemuan sebelumnya. Implementasi tampilan output tersebut akan terlihat seperti Gambar 0.16.

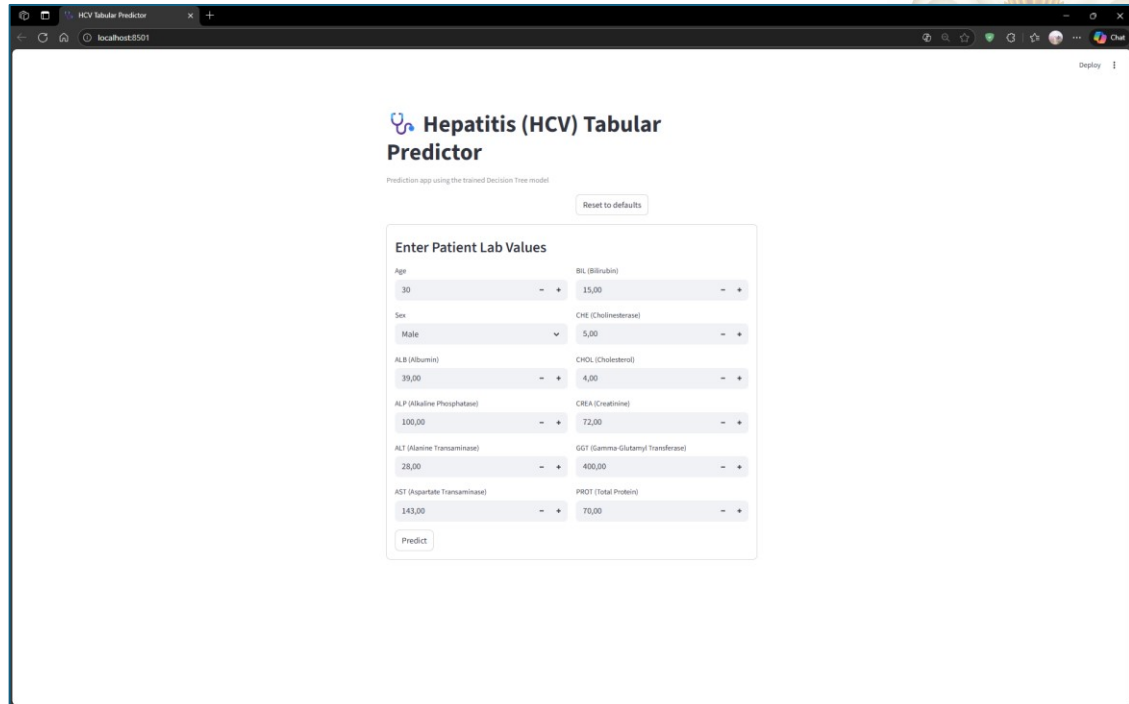


```

1  if submitted:
2      with st.spinner("Loading model and predicting..."):
3          model = load_model(MODEL_PATH)
4          sex_enc = _encode_sex(sex)
5
6          columns_order = ['Sex', 'Age', 'ALB', 'ALP', 'ALT', 'AST', 'BIL', 'CHE', 'CHOL', 'CREA', 'GGT', 'PROT']
7          df_input = pd.DataFrame(columns=columns_order)
8          df_input.loc[0] = [
9              sex_enc,
10             _to_float(age),
11             _to_float(alb),
12             _to_float(alp),
13             _to_float(alt),
14             _to_float(ast),
15             _to_float(bil),
16             _to_float(che),
17             _to_float(chol),
18             _to_float(crea),
19             _to_float(ggt),
20             _to_float(prot),
21         ]
22
23         result = model.predict(df_input)
24         int_result = int(result[0])
25         if int_result == 0:
26             decision = "Suspect Blood Donor"
27         elif int_result == 1:
28             decision = "Hepatitis"
29         elif int_result == 2:
30             decision = "Fibrosis"
31         elif int_result == 3:
32             decision = "Cirrhosis"
33         else:
34             decision = "Not defined"
35
36         st.success("Prediction complete")
37
38         st.metric(label="Predicted Status", value=decision)
39
40         with st.expander("Input Summary"):
41             st.write({
42                 "age": age,
43                 "sex": sex,
44                 "alb": alb,
45                 "alp": alp,
46                 "alt": alt,
47                 "ast": ast,
48                 "bil": bil,
49                 "che": che,
50                 "chol": chol,
51                 "crea": crea,
52                 "ggt": ggt,
53                 "prot": prot,
54             })

```

Gambar 0.14 Halaman Output Prediksi

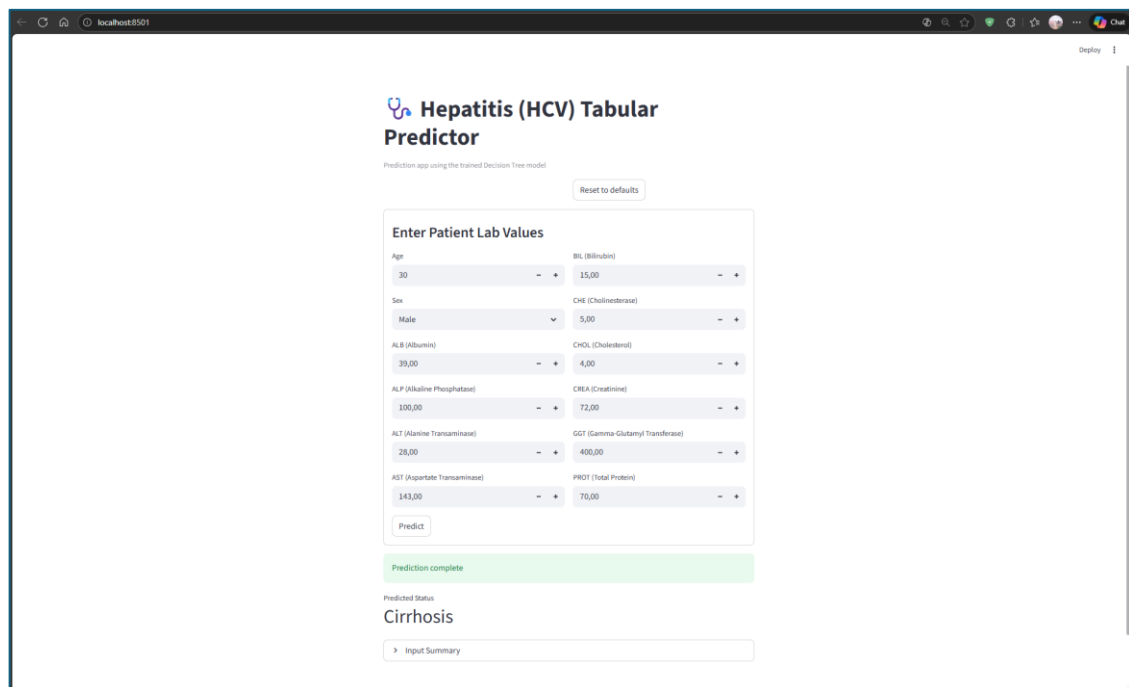


The screenshot shows a web browser window with the URL `localhost:8501`. The page title is "Hepatitis (HCV) Tabular Predictor". Below the title, it says "Prediction app using the trained Decision Tree model". There is a "Reset to defaults" button. The main section is titled "Enter Patient Lab Values" and contains a form with the following fields:

Field	Value
Age	30
Sex	Male
ALB (Albumin)	39,00
ALP (Alkaline Phosphatase)	100,00
ALT (Alanine Transaminase)	28,00
AST (Aspartate Transaminase)	143,00
BL (Bilirubin)	15,00
CHL (Cholesterol)	4,00
CREA (Creatinine)	72,00
GGT (Gamma Glutamyl Transferase)	400,00
PROT (Total Protein)	70,00

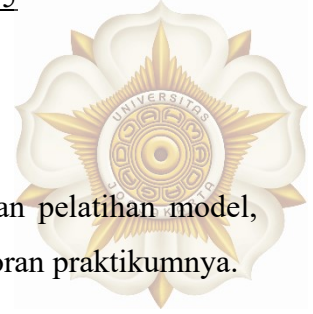
There is a "Predict" button at the bottom of the form.

Gambar 0.15 Halaman Input



The screenshot shows the same web browser window as before, but now the "Predict" button has been clicked. A green bar at the bottom of the form area says "Prediction complete". Below this, the "Predicted Status" is displayed as "Cirrhosis". There is a link to "Input Summary" below the predicted status.

Gambar 0.16 Halaman Input dan Output



11.5. TUGAS & ANALISIS

Silakan menggunakan dataset dari proyek kelompok untuk melakukan pelatihan model, kemudian buatlah dashboard dengan Streamlit. Setelah itu, buatlah laporan praktikumnya.

11.6. REFERENSI

<https://docs.streamlit.io/>

<https://www.python.org/doc/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

<https://numpy.org/>

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.